Glossary: plain-language definitions

Keep this page open beside the workbook. Each has a picture from everyday life.

Word	Plain meaning	Everyday picture / where you meet it
test	A tiny program that checks one thing about another program and answers pass or fail.	Tasting the soup before serving. Workbook Part A1.
unit test	A test of one small piece, usually one function.	Pressing the smoke-alarm button, not burning the house down. Part A1.
pytest	The tool that finds your tests, runs them, and reports.	The referee. Part A2.
assert	A line in a test that says "I claim this is true." If it is false, the test fails.	`assert length_label(179) == "short"` Step 04a.
red	At least one test failed. Normal, and often good: it proves the test can notice a problem.	Part A2.
green	Every test passed.	Part A2.
Track / listing	A *Track* is one playlist entry (id, title, artist, length). A *listing* is the same thing; one song can have several listings.	`playlist/track.py`.
function	A named recipe: give it an input, it gives back an answer.	`length_label(90)`.
look alike	Our first, rough rule for a duplicate: same artist and lengths within 5 seconds.	Step 06a.
matrix	A grid of rows and columns; here, 1 where a pair is in the relation and 0 where not. The *diagonal* runs top-left to bottom-right.	Step 01.
PREDICT / RUN / EXPLAIN	The three beats of every step: guess first, run the test, say why in your own words.	Part A3.
stub	A placeholder function that exists but does not do the real job yet (for example, it returns `None`).	A cardboard cutout of a door. Step 04a.
fixture	Made-up sample data that tests reuse. (pytest uses the word more broadly; here it is only the sample tracks in `tests/fixtures.py`.)	The same props on the stage for every scene. Part A4.
bin (partition cell)	A group of inputs that the function treats the same way. A **partition** cuts all inputs into bins with no overlap and nothing left out.	Postage by weight bracket. Beat 1.
boundary	The values right on either side of the edge between two bins. Test both sides.	A parcel of exactly 1 lb: which bracket? Step 04a.
relation	A yes/no question about two things.	"Are these two listings the same song?" "Does A come before B?" Step 05.
property	A rule that must hold for *every* item or pair, not just one example.	"A track is a duplicate of itself." Step 05, 07, 10a, 11a.
reflexive	Every item is related to itself.	Everyone sits at their own table.
symmetric	If a is related to b, then b is related to a.	"Is the sibling of." (Not: "is the boss of.") Step 10a.
antisymmetric	If a is related to b *and* b is related to a, then a and b are the same item.	A photo finish with a tie breaks this. Step 11a.

Word	Plain meaning	Everyday picture / where you meet it
transitive	If a relates to b and b relates to c, then a relates to c.	"Sits near" in a row of chairs is *not* transitive. Step 07.
equivalence relation	A relation that is reflexive, symmetric and transitive. It sorts everything into groups that do not overlap: "counts as the same."	Same table number. "Same label." Step 05, 08.
partial order	A relation that is reflexive, antisymmetric and transitive: "comes before or is equal to." Some pairs may not be comparable at all.	Socks before shoes; shirt before jacket. Step 11a, 13a.
witness	A concrete example that shows a claim is true or false.	Three actual tracks with actual lengths. Step 07.
counterexample	A witness that *breaks* a rule that was supposed to always hold.	200 s, 204 s, 208 s with a gap of 5. Step 07.
normalize	Rewrite things into one standard form so equal-looking things become equal.	Lowercase, drop the "(Remastered)". Step 08.
key	The standard form that normalizing produces. Two things are the same exactly when their keys are equal.	`("lantern hill", "slow orbit")`. Step 08.
characterization test	A test written to record that something already works. It starts green.	A photo of the room before you rearrange it. Step 05.
cluster / chain	Items joined into one group by a chain of matches (friend of a friend).	Beat 3.
depends on / needs	"X depends on Y" means X uses Y, so Y must work first.	Shoes need socks. Beat 7.
leaves first	Test the pieces that need nothing else first, then the pieces that rely on them.	Beat 7.
cycle	A loop of "needs": A needs B and B needs A. There is no first thing to test.	The key is locked inside the room. Step 14a.
AI proposed / verified	A suggestion from Claude is only a *proposal* until you run it or check it by hand. That check makes it *verified*.	The honesty log, C2.
tuple	A short fixed list written `(a, b, c)`. The witness finders return one.	Steps 02, 07.
raise / ValueError	To stop with an error because the input is not allowed. A test can check that the error happens.	`length_label(-1)`. Step 04a.
witness finder	A function in `tests/relations.py` that hunts for a counterexample and returns it (a tuple) or `None` if there is none.	Step 02.
graph (of "needs")	A list saying, for each function, which functions it needs.	Beat 7.
union-find / connected components	A standard way to merge items into groups whenever a chain of matches links them. You do not write it; Claude does, and you check the groups.	Step 09.
depth-first walk	To order things, visit what an item needs first (and what *that* needs), then add the item.	Step 13b.

Word	Plain meaning	Everyday picture / where you meet it
permutations	All the different orders a list can be put in. 6 tracks have 720. Used to check a sort gives one answer.	Step 11a.
canonical key	Another name for the key made by normalizing: one standard form per thing.	Step 08.
ordered pair	Two things where order matters: (Ana, Ben) is not (Ben, Ana).	Beat 0.
TDD (test-driven development)	Write the test first, watch it fail (red), write the code, watch it pass (green).	Whole workbook.
property test	A test that checks a rule over many items, not one example.	Steps 05, 07, 10, 11.
design smell	A sign in the structure (like a cycle) that the design should change.	Step 14.